Lab 02

Objectives:

This lab will enable students to:

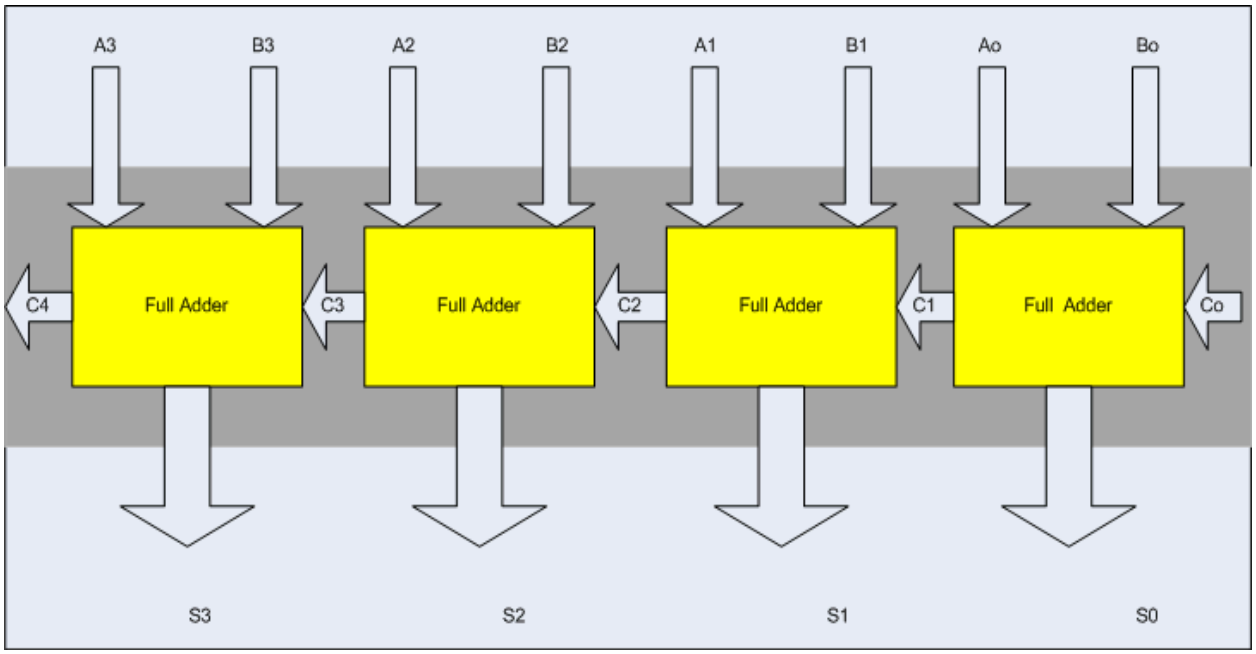
- Learn top down and bottom up design methodologies
- Data flow level modeling

TASK 1:

4 bit Ripple Carry Adder

Block Diagram:

Following is the block diagram of a 4 bit Ripple Carry Adder.



Steps for task 1a:

The following steps should be performed while designing a 4 bit RCA adder

ModelSim:

1- First implement a Full adder using data gate level modeling.
2- Simulate the Full adder with a test bench.
3- Instantiate the Full adder four times and connect the circuit as shown.
4- Now again write a test bench and simulate the 4 bit RCA.

ALGORITHM:

Step 1: Define Basic Logic Gates

1. **Sum Module** (Half Adder XOR Gate)

    - Inputs: A, B

    - Output: S (Sum)
    - Operation: $S = A \oplus B$
2. **Carry Module** (Half Adder AND Gate)

    - Inputs: A, B
    - Output: C (Carry)
    - Operation: $C = A \cdot B$

Step 2: Construct Half Adder (HA)

1. Uses **Sum** and **Carry** modules.
2. Inputs: A, B
3. Outputs: S (Sum), C (Carry)
4. Calls:
    - Sum module to compute sum.
    - Carry module to compute carry.

Step 3: Construct Full Adder (FA)

1. Uses two **Half Adders**.
2. Inputs: A, B, Cin
3. Outputs: S (Sum), C (Carry)
4. Steps:
    - First **HA** computes s1 and c1 for A and B.
    - Second **HA** computes S and c2 for s1 and Cin.
    - OR gate computes C = c1 + c2.

Step 4: Implement 4-bit Ripple Carry Adder (RCA)

1. Inputs:
    - A[3:0] (4-bit operand)
    - B[3:0] (4-bit operand)
    - Cin (Initial carry)
2. Outputs:
    - S[3:0] (4-bit sum result)
    - Cout (Final carry output)
3. Steps:
    - Use **4 Full Adders** in a sequence.
    - Connect carry output of each FA to the carry input of the next FA.
    - Final carry out from FA4 is Cout.

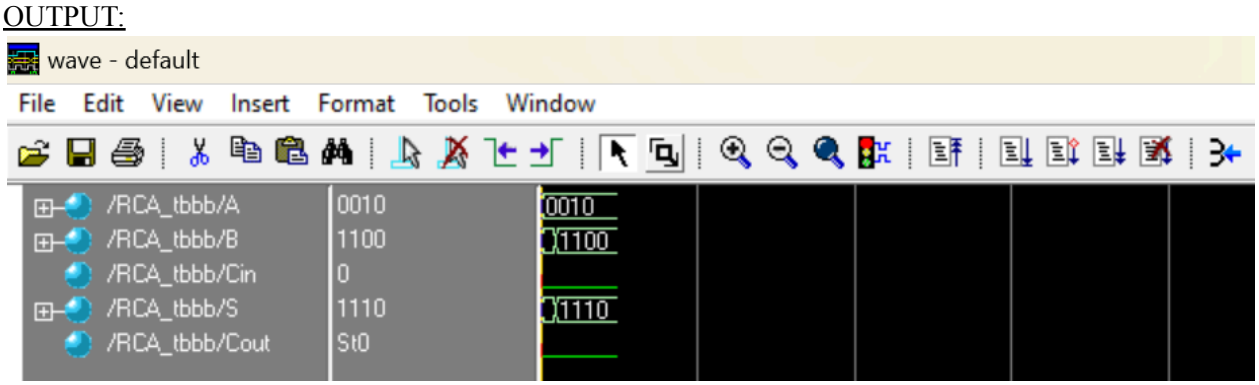
Step 5: Testbench (RCA_tbbb)

1. Define test cases:
    - A = 4'b0010, B = 4'b0010, Cin = 0
    - A = 4'b0010, B = 4'b1100
2. Monitor the sum and carry outputs.
3. Run simulation for verification.

CODE:

```
module Sum(S,A,B);
input A,B;
output S;
xor x(S,A,B);
endmodule

module Carry(C,A,B);
input A,B;
output C;
and a(C,A,B);
endmodule

module HA(S,C,A,B);
input A,B;
output S,C;
Sum s1(S,A,B);
Carry c1(C,A,B);
endmodule

module FA(S,C,A,B,Cin);
input A,B,Cin;
output S,C;
wire s1,c1,c2;
HA h1(s1,c1,A,B);
HA h2(S,c2,Cin,s1);
or o(C,c1,c2);
endmodule

module RCA(S,Cout,A,B,Cin);
input [3:0] A,B;
input Cin;
output [3:0] S;
output Cout;
wire [2:0] c;
FA f1(S[0],c[0],A[0],B[0],Cin);
FA f2(S[1],c[1],A[1],B[1],c[0]);
FA f3(S[2],c[2],A[2],B[2],c[1]);
FA f4(S[3],Cout,A[3],B[3],c[2]);
endmodule

module RCA_tbbb();
reg [3:0] A,B;
reg Cin;
wire [3:0] S;
wire Cout;
RCA dut(S,Cout,A,B,Cin);
initial
begin
A= 4'b0010;
B= 4'b0010;
Cin=0;

#20
A=4'b0010;
B=4'b1100;
end
initial
$monitor("A= %b", A , "B= %b", B, "Sum= %b", S, "Cout= %b", Cout);
endmodule
```

OUTPUT:



<u>Task 1b:</u>

Design the 4 bit full adder using data flow level modeling.

<u>ALGORITHM:</u>

Step 1: Define Half Adder (HA)

1. Inputs: A, B
2. Outputs: S (Sum), C (Carry)
3. Operations:
    - Sum: $S = A \oplus B$
    - Carry: $C = A \cdot B$

Step 2: Define Full Adder (FA)

1. Inputs: A, B, Cin
2. Outputs: S (Sum), C (Carry)
3. Operations:
    - Intermediate sum: $s1 = A \oplus B$
    - Intermediate carry: $c1 = A \cdot B$
    - Final sum: $S = s1 \oplus Cin$
    - Second carry: $c2 = s1 \cdot Cin$
    - Final carry: $C = c1 + c2$

Step 3: Implement 4-bit Ripple Carry Adder (RCA)

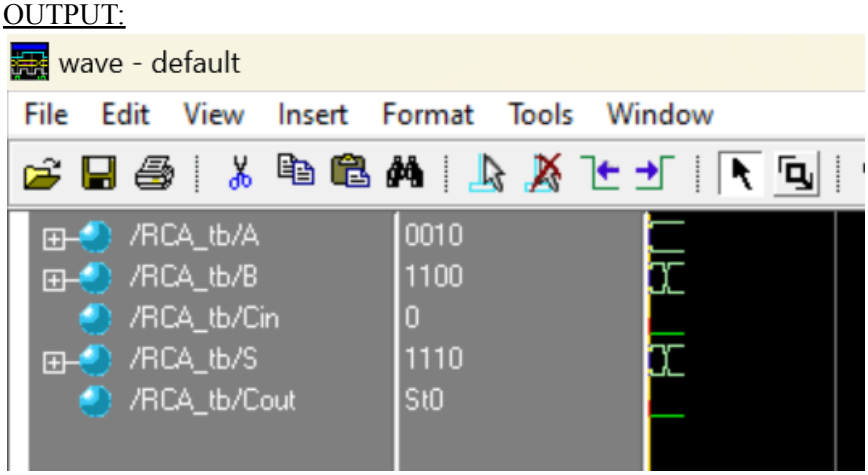
1. Inputs:
    - A[3:0] (4-bit input)
    - B[3:0] (4-bit input)
    - Cin (Initial carry)
2. Outputs:
    - S[3:0] (4-bit sum)
    - Cout (Final carry out)
3. Steps:
    - Use **4 Full Adders (FA)** in sequence.
    - Carry from one FA propagates to the next.
    - Connect the carry-out of the last FA to Cout.

Step 4: Testbench (RCA_tb)

1. Define test cases:
    - A = 4'b0010, B = 4'b0010, Cin = 0
    - A = 4'b0010, B = 4'b1100, Cin = 0
2. Monitor results:
    - Sum output (S)
    - Carry output (Cout)
3. Run simulation and verify outputs.

<u>CODE:</u>

```
module HA(S, C, A, B);
input A, B;
output S, C;
assign S = A ^ B;
assign C = A & B;
endmodule

module FA(S, C, A, B, Cin);
input A, B, Cin;
output S, C;
wire s1, c1, c2;
assign s1 = A ^ B;
assign c1 = A & B;
assign S = s1 ^ Cin;
assign c2 = s1 & Cin;
assign C = c1 | c2;
endmodule

module RCA(S, Cout, A, B, Cin);
input [3:0] A, B;
input Cin;
output [3:0] S;
output Cout;
wire [2:0] c;

FA f1(S[0], c[0], A[0], B[0], Cin);
FA f2(S[1], c[1], A[1], B[1], c[0]);
FA f3(S[2], c[2], A[2], B[2], c[1]);
FA f4(S[3], Cout, A[3], B[3], c[2]);
endmodule
```

```
module RCA_tb();
reg [3:0] A, B;
reg Cin;
wire [3:0] S;
wire Cout;

RCA dut(S, Cout, A, B, Cin);
initial begin
A = 4'b0010; B = 4'b0010; Cin = 0;
#20;
A = 4'b0010; B = 4'b1100; Cin = 0;
#20;

$finish;
end

initial
        $monitor("A= %b, B= %b, Cin= %b, Sum= %b, Cout= %b", A, B, Cin, S, Cout);
endmodule
```

OUTPUT: